Accelerating MoE Token Routing and Dispatch on GPU

Feiyang Wu
feiyang2@andrew.cmu.edu

Yuhang Hu
yuhangh@andrew.cmu.edu

Abstract

We implemented and evaluated a set of optimizations for the Expert Parallelism (EP) layer inside Mixture-of-Experts (MoE) language models, focusing on two bottlenecks: token scatter and the expert feed-forward network (FFN). For scatter, we built five progressively improved CUDA kernels and integrated them into a complete multi-GPU EP prefill and decode pipeline. For the FFN, we evaluated two alternatives to the sequential per-expert cuBLAS approach: a blocksparse kernel and a CUTLASS grouped GEMM. We benchmarked on NVIDIA Tesla V100-SXM2-32GB GPUs at the Pittsburgh Supercomputing Center and on NVIDIA A100-SXM4-40GB GPUs via Modal cloud. Our best scatter kernel reduces scatter latency by 12.7x on V100 for $T = 2048$ tokens, producing a 1.22x end-to-end EP round speedup. On A100, combining CSort scatter, communication overlap, and CUTLASS grouped GEMM reduces 2-GPU Zipf prefill round time from 6.38 ms to 4.62 ms, a 1.38x speedup over the Atomic baseline. At fixed workload $T = 8192$, the optimized 8-GPU pipeline reduces prefill time from 503.4 ms for the 1-GPU Atomic baseline to 100.2 ms under uniform routing, a 5.03x end-to-end speedup. Under Zipf routing, the same comparison reaches only 1.92x speedup, showing that hot-expert imbalance is the main scaling bottleneck. Blocksparse FFN was slower than packed cuBLAS in all configurations we tested. Pipeline overlap is most useful in decode workloads, where it improves latency by 10–27%, but its benefit in prefill depends on whether communication can be hidden behind enough local computation.

1 Background

1.1 Mixture-of-Experts and Expert Parallelism

A Mixture-of-Experts (MoE) layer replaces a standard feed-forward network with a set of E expert networks and a gating module that sends each token to a small subset of these experts at runtime [3]. With top-K routing, each token is assigned to K experts out of E, where typically $K \ll E$. Only the chosen experts run for that token, which keeps per-token compute roughly constant even as the number of experts grows. When the model is small enough to fit on one GPU, all E experts live on the same device and the layer is dominated by the FFN matrix multiplications. When the model grows beyond single-GPU memory, Expert Parallelism (EP) splits the experts across N GPUs so that each GPU holds E/N of them [2]. This shrinks per-GPU FFN cost roughly linearly in N, but it also means that most tokens now live on a different GPU than their assigned expert. Before expert computation can begin, tokens must be rearranged and packed into contiguous memory according to their assigned experts. The cost of this movement does not shrink with N in the same way the FFN does, so as N grows, token movement becomes the new bottleneck. Figure 1 shows the five stages that a single MoE layer goes through under EP for a batch of T tokens.

In stage 1 (scatter), each GPU reorders its local token embeddings by expert destination into a contiguous packed buffer. In stage 2 (dispatch), each GPU sends each contiguous expert-block to the GPU that owns that expert, using NCCL AllToAll. In stage 3 (FFN), each GPU runs a local FFN

using cuBLAS. In stage 4 (combine), the processed tokens are sent back to the GPU they originally came from, again using NCCL AllToAll. In stage 5 (merge), each token's K expert outputs are combined using the gate weights produced during routing.

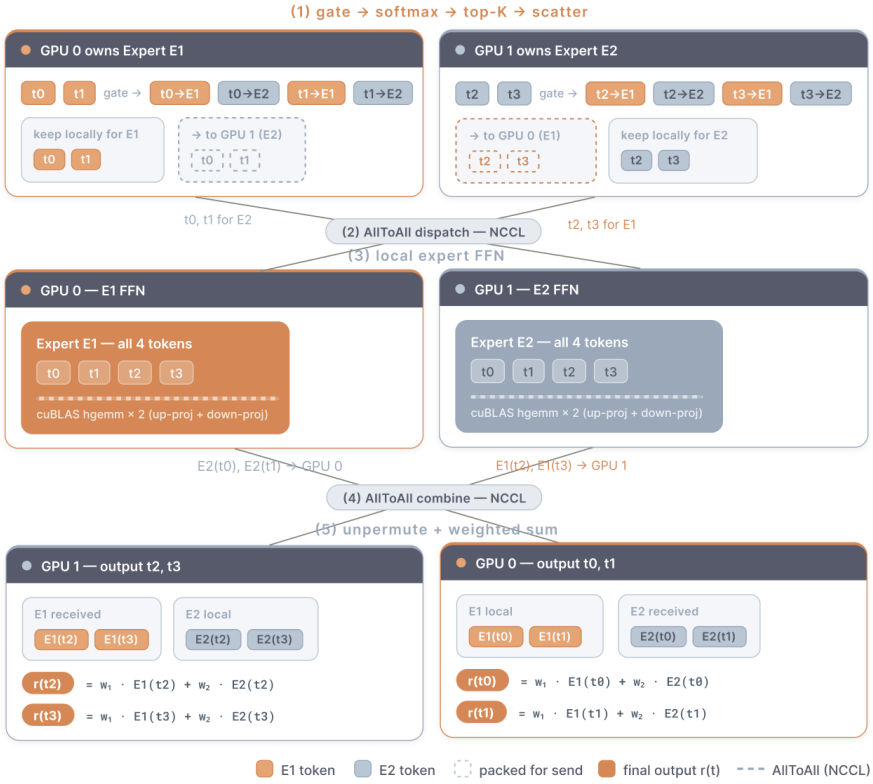


Figure 1: The five-stage Expert Parallelism prefill pipeline. Scatter (stage 1) packs token embeddings into expert-contiguous order before the AllToAll dispatch step.

1.2 Key Data Structures

The scatter stage operates on four input tensors and produces three output tensors. Table 1 lists them along with their shapes.

Table 1: Tensors involved in the scatter stage. T is the number of tokens on this GPU, K is the top-K value, E is the total number of experts, and d is the model dimension. In our experiments T ranges from 16 to 8192, $K = 4$, $E = 64$, and $d = 2048$.

Tensor	Shape	Dtype	Role
`embeddings`	$[T, d]$	fp16	Token embeddings to be packed
`assignments`	$[T, K]$	int32	Expert IDs from the gate
`weights`	$[T, K]$	fp32	Gate weights for output merging
`expert_count`	$[E]$	int32	Tokens routed to each expert
`expert_start`	$[E]$	int32	Prefix sum of `expert_count`
`packed`	$[TK, d]$	fp16	Reordered embeddings, expert-contiguous
`perm`	$[TK]$	int32	Source token index for each packed slot
`packed_w`	$[TK]$	fp32	Gate weight for each packed slot

The **packed** buffer is the central data structure. Tokens routed to expert e are gathered into a contiguous slice, so the receiving GPU can run its FFN as a single matrix multiply on a dense $[n_e, d]$ block and send the result back in one NCCL call. The permutation array records where each

token came from, so the combine stage can write each output back to its original position. Figure 2 illustrates how a small example batch is reordered. Each input row in `embeddings` is copied to a slot in `packed` whose position depends on the runtime expert assignment.

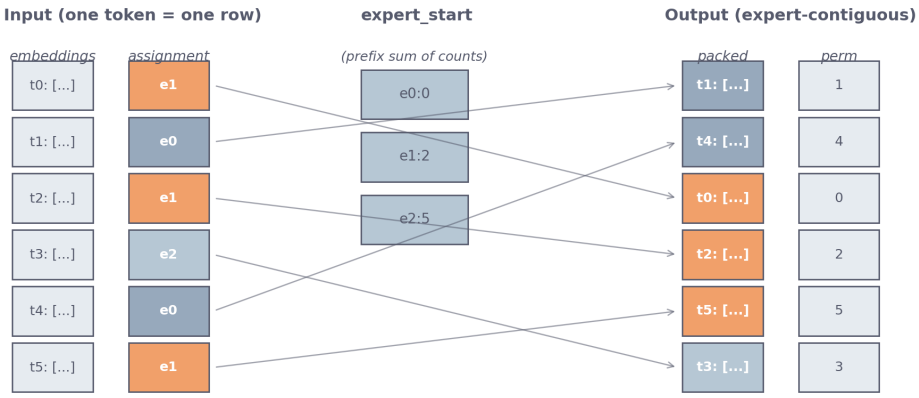


Figure 2: Scatter reorders token embeddings so that all tokens going to the same expert end up in a contiguous slice of the output buffer. The destination of each row is determined by the gate at runtime.

1.3 Workload Analysis

The scatter stage has three sources of parallelism as shown in Figure 1. The first is **across tokens**. The TK token-expert pairs are independent in the sense that the expert assignment for each token has already been decided by the gate. The only dependency is on the per-expert counter that determines packed-buffer position. The second source is **within a row**. Each row has d fp16 values, which equals $d/8$ chunks of 16 bytes when read as `uint4`. A warp's 32 lanes cooperate on this row, with each lane handling $d/256$ chunks in a stride-32 loop. On one hand, each lane uses 16-byte (`uint4`) loads and stores instead of 2-byte fp16 ones, which cuts the instruction count by a factor of 8. On the other hand, in any single iteration of the loop the 32 lanes access 32 consecutive 16-byte chunks, so the hardware merges their loads into wide HBM transactions and merges their stores in the same way. The only requirement is that d be a multiple of 8 so the row aligns to `uint4` boundaries; this holds for all model sizes we test, where $d = 2048$ matches Qwen1.5-MoE-A2.7B and gives 256 chunks per row, 8 chunks per lane. The combination of vectorization and coalescing brings scatter close to the bandwidth ceiling for a pure copy. The third source is at the **warp level**. Computing the destination slot s for each row requires an atomic increment on `expert_count`$[e]$, because multiple warps processing different tokens may target the same expert region. In the per-thread design, each lane handles its own token and does its own atomic, so a warp issues 32 atomics. In this project, one warp is dedicated to one (token, expert) pair: lane 0 alone performs the atomic, and the resulting slot index is broadcast to the other 31 lanes through a warp shuffle. The other lanes then participate in copying the row.

Locality is good for reads and writes individually but poor across rows. Reading row t from `embeddings` is a contiguous d-element load. Writing row t to `packed` is a contiguous d-element store. But two adjacent threads handling two adjacent tokens may write to two completely different regions of `packed` because their tokens went to different experts. The access pattern within a row is dense and regular, but the mapping from input rows to output rows is data-dependent.

The FFN stage, by contrast, has no such irregularity once the data is packed. Each per-expert sub-buffer is a dense $[n_e, d] \times [d, d_{\text{ffn}}]$ matrix multiplication, which maps cleanly onto Tensor Cores. The challenge in the FFN is different. With $E = 64$ experts on one GPU, the standard approach issues 64 separate cuBLAS calls, one per expert. Each call has fixed launch overhead. Under skewed routing, many experts receive only a handful of tokens, and the per-call overhead dominates the actual matmul time. A grouped GEMM kernel that handles all 64 experts in one launch can recover this overhead.

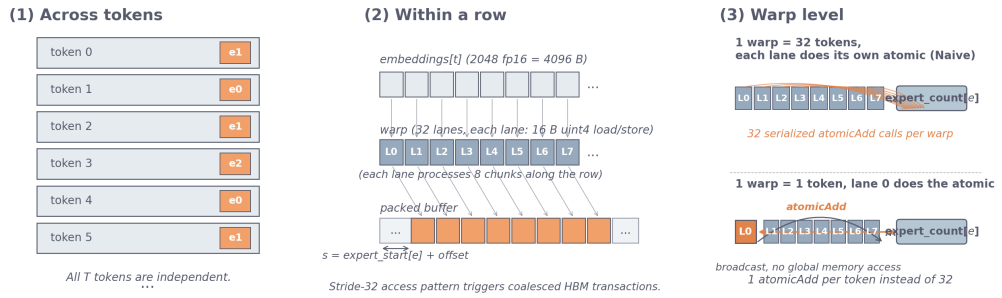


Figure 3: Three sources of parallelism in scatter.

These two bottlenecks are the focus of the rest of this paper. Section 2 describes five scatter kernels that together explore the design space of conflict resolution, vectorization, and warp-level work distribution, along with two FFN alternatives.

2 Approach

2.1 Technologies and Target Platforms

We implemented all kernels in CUDA C++. We used CUB for the radix sort and exclusive prefix sum primitives in the sort-based scatter kernels, CUTLASS for the grouped GEMM, and NCCL for inter-GPU point-to-point send/recv and AllGather.

We targeted two GPU platforms. The first is NVIDIA Tesla V100-SXM2-32GB at PSC Bridges-2, running CUDA 12.6.1 with CUB and NCCL. The second is NVIDIA A100-SXM4-40GB through Modal cloud, running CUDA 12.4.1 with CUTLASS 3.5.1 and NCCL. The two platforms exercise different parts of the design space. V100 is more strictly bandwidth-bound, so memory-movement optimizations show up clearly in scatter latency. A100 has more compute headroom and faster Tensor Cores, which makes the FFN launch overhead more visible and motivates the grouped GEMM experiments.

2.2 Atomic Baseline

The baseline pipeline runs a count kernel that builds a shared-memory histogram of expert assignments and flushes it to a global `expert_count` array, then a CUB exclusive prefix sum to produce `expert_start`, and finally a scatter kernel where each thread handles one (token, expert) pair independently. This design has two performance bottlenecks. **First**, every thread calls `atomicAdd` on the same per-expert counter, so threads assigned to the same hot expert must serialize. Under Zipf distribution this is severe. **Second**, the element-by-element copy loop is not vectorized, and the 32 threads in a warp write to 32 different row addresses, so the GPU cannot merge their stores into a single wide memory transaction.

2.3 Scatter Optimization

We built four improved kernels on top of the atomic baseline. Figure 4 shows how each kernel maps the work onto GPU execution units: the unit of work is one row of the `packed` buffer, which is assigned to either a thread block (Sort) or a single warp of 32 lanes (Warp, Vec, CSort), with lane i handling chunks $i, i+32, i+64, \ldots$ of the row in a stride-32 loop. The figure also shows which rows each unit writes and how it handles the per-expert counter.

Sort. The sort strategy replaces the scatter kernel with a CUB radix sort on expert IDs (using only $\lceil \log_2 E \rceil$ bits), followed by a gather kernel that assigns one 256-thread block to each output row. Because the sort pre-determines every slot's destination, there are no atomics in the gather phase. All 256 threads in a block write consecutive elements of the same output row, so their stores coalesce into 128-byte transactions. The trade-off is two extra HBM passes for the sort's temporary key and value arrays.

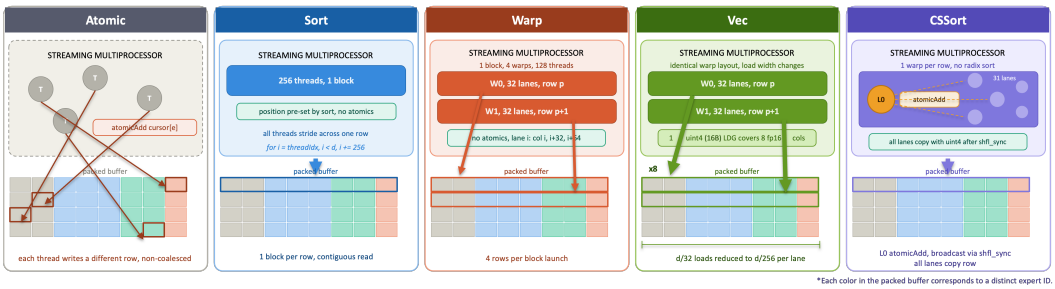


Figure 4: Five CUDA kernel strategies for packing rows into a contiguous buffer.

Warp. The warp strategy keeps the same sort pipeline but changes only the gather kernel geometry. Instead of one block per row, it assigns one warp (32 lanes) per row, with four warps sharing a single block launch. This produces four times fewer block launches than Sort, reducing launch overhead. The stride-32 access pattern remains coalesced because all 32 lanes in a warp read and write consecutive elements within one warp step.

Vec. The vec strategy is identical to Warp but reinterprets the fp16 embedding array as `uint4` (16 bytes, holding 8 fp16 values) so that each load/store instruction moves 16 bytes instead of 2. This reduces the number of load and store instructions per row by a factor of 8, freeing SM issue slots. It also fuses the two-kernel bounds-plus-count sequence into a single kernel, saving one kernel launch. The requirement is that `d_model` is a multiple of 8.

CSort. CSort avoids the radix sort entirely, eliminating its two extra HBM passes. It reuses the same count kernel and prefix sum as the baseline to build `expert_start`, then runs a single `place_gather` kernel where lane 0 of each warp performs the atomic and broadcasts the result. The key difference from the atomic baseline is that the atomic runs once per warp rather than once per thread, cutting atomic operations by a factor of 32. For the hottest expert under Zipf, this means roughly 50 sequential atomic operations instead of over 1000. The `__shfl_sync` call broadcasts the slot position to all 32 lanes using warp registers with no global memory access. All lanes then copy the embedding row with the same vectorized `uint4` pattern as Vec.

Iteration. The four kernels were developed in the order presented, each step driven by the previous one's bottleneck. Atomic serializes threads that share the same expert counter, every thread issues its own `atomicAdd`, and the hardware queues them at the L2 cache. Sort makes the gather phase lock-free by fixing every slot's destination ahead of time, but launches one block per row. Warp shrinks the per-row unit to a single warp, cutting block launches by 4x. Vec then widens each lane's memory instructions from 2-byte fp16 to 16-byte `uint4`, reducing instruction count by 8x. At this point the radix sort's two extra HBM passes became the bottleneck, so CSort drops the sort, keeps the warp-per-row `uint4` copy from Vec, and reserves slots through a single per-warp atomic broadcast via `__shfl_sync`.

2.4 FFN Optimizations

Each expert's FFN follows the standard two-layer design used in Transformer feed-forward blocks [1], with two weight matrices $W_1 \in \mathbb{R}^{d \times d_{\text{ffn}}}$ and $W_2 \in \mathbb{R}^{d_{\text{ffn}} \times d}$ separated by a nonlinearity:

$$FFN_e(x) = W_2 \cdot \sigma(W_1 \cdot x), \tag{1}$$

where σ is a pointwise activation. We refer to the W_1 step as the up-projection (mapping $d \to d_{\text{ffn}}$) and the W_2 step as the down-projection (mapping $d_{\text{ffn}} \to d$). In our experiments $d = 2048$ and $d_{\text{ffn}} = 8192$, matching Qwen1.5-MoE-A2.7B.

The packed FFN baseline issues one cuBLAS HGEMM call per expert for each of the two matmuls, giving $2E = 128$ sequential calls per FFN pass with $E = 64$. The launch overhead of these calls dominates the compute time when expert groups are small. Both alternatives we explored target the up-projection, since it is the more memory-traffic-intensive of the two matmuls and the one that

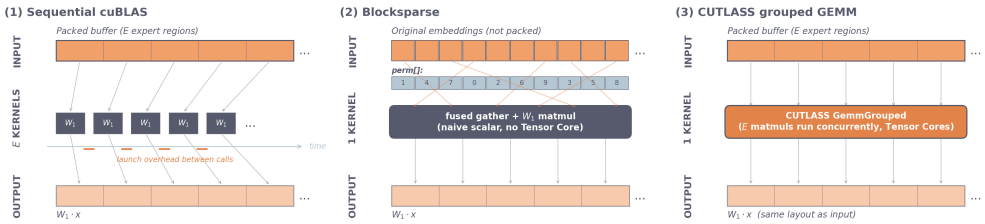


Figure 5: Three implementations of the up-projection.

benefits most from fused-gather techniques. Figure 5 compares the three implementations of the up-projection.

Blocksparse. The blocksparse approach fuses the scatter step and the up-projection into a single kernel, following the Megablocks design [1]. Instead of materializing the packed buffer before W_1, the kernel reads token rows directly from the original `embeddings` through the sorted index produced during routing, and computes $embeddings[perm] \cdot W_1$ in place. This saves one full HBM read-and-write of the packed buffer.

Grouped GEMM. We replaced the E sequential cuBLAS calls for W_1 with a single CUTLASS `GemmGrouped` kernel that handles all expert up-projections in one launch. The kernel takes a list of per-expert pointer-and-shape tuples, and the GroupedGEMM scheduler runs the E matmuls concurrently on different SMs inside one launch. Tensor Cores are used at full throughput, since each per-expert input slice is contiguous in the packed buffer.

2.5 Origin of Code

All kernels were written from scratch. We studied the open-source implementations of ScatterMoE [4], MegaBlocks [1], DeepEP [5], and SGLang/vLLM as design references for scatter and dispatch patterns.

3 Results

3.1 Experimental Setup

All results are medians over multiple repeated runs after warmup, averaged across participating GPUs. Model dimensions match Qwen1.5-MoE-A2.7B: $E = 64$, $K = 4$, $d_{\text{model}} = 2048$, $d_{\text{ffn}} = 8192$. We used two expert assignment distributions. Uniform assigns tokens with near-equal load across experts. Zipf generates roughly 9–11x load imbalance, modeling skewed real-world inference traffic.

We evaluated 2-GPU, 4-GPU, and 8-GPU EP configurations using NCCL point-to-point send and recv for the dispatch and combine stages, with an AllGather before each dispatch to collect per-expert token counts so every GPU knows how many tokens to expect from each peer. Within each configuration we tested two scheduling modes. In serial mode each pipeline stage runs to completion before the next begins. In overlap mode, scatter and FFN run on one CUDA stream while NCCL AllToAll runs on a second stream, with event-based synchronization between them.

3.2 Scatter Latency on V100

Table 2 shows per-round times for all five scatter strategies at 2 GPUs with $T = 2048$ under uniform expert assignment on V100.

Table 2: V100, 2 GPU, T=2048, Uniform. Times in ms per round.

Strategy	Scatter	Dispatch	Combine	FFN	Round
Atomic	1.281	0.740	0.688	8.860	11.569
Sort	0.121	0.616	0.600	8.164	9.500
Warp	0.106	0.637	0.603	8.134	9.481
Vec	0.102	0.629	0.603	8.145	9.480
CSort	0.101	0.642	0.603	8.157	9.504

The sort-based strategies (Sort, Warp, Vec, CSort) reduce scatter from 1.28 ms to about 0.10 ms, a 12.6x improvement. End-to-end round time drops from 11.57 ms to about 9.48 ms, a 1.22x speedup. The EP overhead fraction (scatter, dispatch and combine) falls from 23.4% to 14.1%. The limited end-to-end gain relative to the large scatter improvement is a consequence of Amdahl's Law: FFN accounts for 77–86% of round time, so scatter optimization alone cannot produce large total speedups.

Table 3 shows scatter latency across different token counts and distributions at 2 GPUs on V100.

Table 3: V100, 2 GPU, scatter latency (ms) across workloads.

T	Dist	Atomic	Sort	Warp	Vec	CSort
512	Uniform	1.172	0.080	0.076	0.078	0.075
2048	Uniform	1.096	0.119	0.107	0.104	0.097
8192	Uniform	2.634	0.319	0.290	0.285	0.188
512	Zipf	1.083	0.080	0.079	0.075	0.077
2048	Zipf	1.110	0.123	0.110	0.108	0.104
8192	Zipf	2.042	0.321	0.295	0.290	0.191

Table 3 reveals several patterns across token counts and distributions. Atomic latency does not grow linearly with T in the same way as the sort-based strategies. Under uniform distribution, Atomic increases from 1.172 ms at T=512 to 2.634 ms at T=8192, a factor of 2.2x. Sort grows from 0.080 ms to 0.319 ms over the same range, a factor of 4x. The sort-based strategies scale closer to linearly in T because their bottleneck is HBM bandwidth, which grows with the amount of data moved. Atomic latency is partially capped by GPU scheduling throughput, so its absolute value grows more slowly despite its much higher baseline.

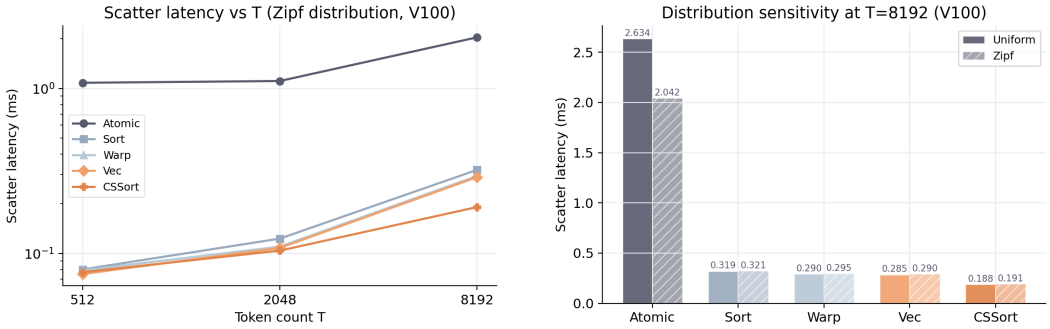


Figure 6: Scatter latency across workloads on V100 with 2 GPUs.

Sort, Warp, Vec, and CSort are all insensitive to expert load distribution as shown in figure 6. At T=8192, Sort gives 0.319 ms under uniform and 0.321 ms under Zipf, a difference of under 1%. This is expected because the radix sort pre-assigns every slot before any data moves, so the gather phase carries no dependence on how tokens are distributed across experts. Atomic shows the opposite behavior: at T=8192, its uniform latency (2.634 ms) is higher than its Zipf latency (2.042 ms). Under uniform distribution all 64 experts receive roughly equal token counts, so atomic contention is spread across all expert counters simultaneously. Under Zipf, most experts receive very few tokens and only a handful of counters are heavily contested, which slightly reduces the total serialization pressure.

3.3 Blocksparse FFN on A100

Table 4 shows the performance of the blocksparse kernel compared to packed cuBLAS on A100 at several token counts.

Table 4: A100, 1 GPU, blocksparse vs packed cuBLAS FFN. Times in ms.

T	Dist	Packed	Blocksparse	Ratio
16	Uniform	1.195	1.690	0.71x
64	Uniform	1.958	4.620	0.42x
256	Uniform	2.007	14.459	0.14x
16	Zipf	0.985	1.620	0.61x
64	Zipf	1.597	4.554	0.35x
256	Zipf	1.999	14.640	0.14x
2048	Zipf	3.424	106.842	0.03x

Blocksparse is always slower than packed cuBLAS by a large margin, growing worse as T increases. At $T = 16$ the gap is only 1.4–1.6x, but by $T = 256$ blocksparse is 7x slower and at $T = 2048$ Zipf it takes 107 ms versus 3.4 ms for cuBLAS. The $T = 2048$ uniform case also triggered an illegal memory access and could not be measured. The blocksparse kernel achieves only 1.3–2.6 TFLOPS across all configurations while cuBLAS reaches 1.8–80 TFLOPS by keeping the Tensor Cores fed with contiguous data. Skipping the scatter step does not compensate for this throughput loss.

3.4 Grouped GEMM FFN on A100

Table 5 shows the best CUTLASS grouped GEMM tile configurations compared to sequential cuBLAS for both distributions.

Table 5: A100, 1 GPU, grouped GEMM tile sweep, T=2048. Times in ms, 64 experts.

Tile	Uniform		Zipf	
	cuBLAS	Grouped	cuBLAS	Grouped
64×64×32	2.32	4.13 (0.56x)	2.65	3.79 (0.70x)
128×128×32	2.32	2.51 (0.92x)	2.64	2.37 (1.11x)
256×128×32	2.32	2.05 (1.13x)	2.65	2.51 (1.06x)
128×256×32	2.32	2.18 (1.07x)	2.65	2.17 (1.22x)

Small tiles are slower because each CUTLASS kernel launch has higher overhead than a cuBLAS call, and small tiles do not amortize this cost. With the $128 \times 256 \times 32$ tile, grouped GEMM achieves 1.07x speedup over cuBLAS on uniform distribution and 1.22x on Zipf. The Zipf advantage is larger because sequential cuBLAS calls on tiny expert groups leave the GPU underutilized between calls, while a single grouped kernel launch is more efficient in that case.

3.5 End-to-End Prefill on A100 with Grouped FFN

Table 6 shows end-to-end prefill results on 2 A100 GPUs, $T = 2048$, 32 rounds. The grouped FFN rows use CUTLASS 128×256×32 tiles, which had the best Zipf performance.

Table 6: A100, 2 GPU, T=2048, 32 rounds, avg ms per round.

Strategy	FFN	Dist	Overlap	Avg/round
Atomic	cuBLAS	Uniform	No	6.620
CSort	cuBLAS	Uniform	No	5.575
CSort	cuBLAS	Uniform	Yes	5.223
Atomic	cuBLAS	Zipf	No	7.347
CSort	cuBLAS	Zipf	No	6.379
CSort	cuBLAS	Zipf	Yes	5.962
CSort	Grouped	Zipf	Yes	4.620

Under uniform assignment, CSort with cuBLAS already improves on Atomic by 1.19x. Enabling overlap adds a further 6% improvement, reaching 5.22 ms.

Under Zipf, CSort with cuBLAS also improves over Atomic, reducing latency from 7.35 ms to 6.38 ms (1.15x). Adding overlap brings a further 6.5% reduction to 5.96 ms. However, the gains from scatter optimization alone are smaller than under uniform, because load imbalance makes the sequential cuBLAS FFN calls less efficient under Zipf. Switching to CUTLASS grouped GEMM with overlap together reaches 4.62 ms, a 1.59x speedup over the Atomic baseline and a 1.38x speedup over CSort with cuBLAS alone. This shows that under Zipf, scatter optimization and FFN optimization are both necessary to achieve the full benefit.

3.6 End-to-End Speedup vs Naive Single-GPU

To measure true multi-GPU acceleration rather than weak-scaling throughput growth, we ran a fixed-workload strong-scaling experiment on A100. All configurations use the same prefill workload: $T = 8192$ tokens and 32 rounds. Speedup is measured against the 1-GPU Atomic baseline, which uses Atomic scatter and sequential per-expert cuBLAS FFN. The optimized pipeline uses CSort scatter, two-stream overlap, and CUTLASS grouped GEMM FFN.

Table 7: A100 strong scaling at fixed workload, T=8192, 32 rounds. Speedup is relative to the 1-GPU Atomic baseline for the same routing distribution.

Dist	GPUs	Total (ms)	Avg/round (ms)	Speedup
Uniform	1	350.913	10.966	1.43x
Uniform	2	213.678	6.677	2.36x
Uniform	4	169.687	5.303	2.97x
Uniform	8	100.154	3.130	5.03x
Zipf	1	356.476	11.140	1.34x
Zipf	2	334.424	10.451	1.43x
Zipf	4	288.912	9.029	1.66x
Zipf	8	249.248	7.789	1.92x

Under uniform routing, the optimized pipeline reduces total prefill time from 350.9 ms on 1 GPU to 100.2 ms on 8 GPUs. Relative to the 1-GPU Atomic baseline of 503.4 ms, this is a 5.03x end-to-end speedup on the same workload. This confirms that the improvement is not only a weak-scaling effect from processing more tokens; the same $T = 8192$ batch becomes substantially faster when expert computation is distributed across GPUs and the routing/FFN overheads are reduced.

The scaling is still sublinear compared with the ideal 8x curve. The main reason is that EP introduces work that does not shrink linearly with the number of GPUs. Scatter still has to move all TK token-expert rows into expert-contiguous order, dispatch and combine require cross-GPU communication, and the current overlap implementation still needs a CPU synchronization point after AllGather to compute message sizes before launching NCCL sends and receives. As the per-GPU FFN work shrinks, these routing and communication costs become a larger fraction of the round time.

Zipf routing exposes a different bottleneck. With skewed expert assignments, some experts receive many more tokens than others. Since experts are partitioned across GPUs, the GPU owning a hot

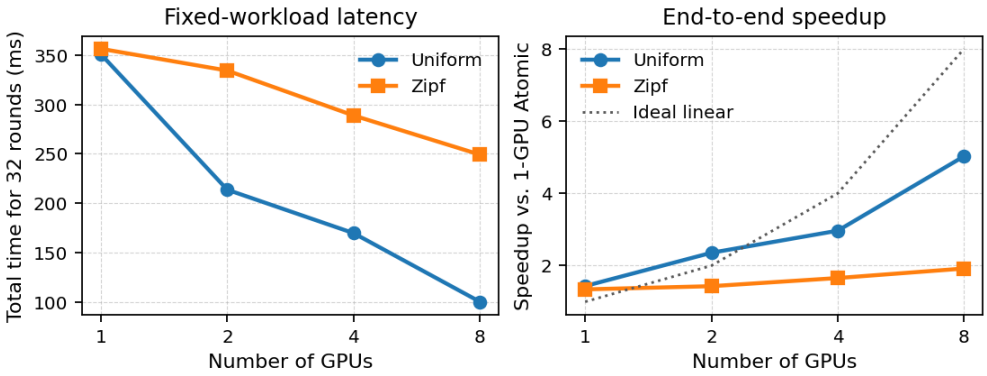


Figure 7: Strong scaling on A100 for a fixed prefill workload, T=8192 and 32 rounds. Left: end-to-end latency decreases as the optimized EP pipeline uses more GPUs. Right: speedup relative to the 1-GPU Atomic baseline. Uniform routing scales much better than Zipf routing because the expert load is balanced across GPUs.

expert becomes a straggler, and the whole MoE round must wait for that tail. This is why the Zipf speedup only reaches 1.92x at 8 GPUs, compared with 5.03x under uniform routing. CSort reduces scatter overhead and grouped GEMM reduces small-GEMM launch overhead, but neither can fully remove the load imbalance created by the routing distribution.

3.7 Decode Workload on A100

Decode inference processes a small number of new tokens per step (typically 1–256) over many rounds. Table 8 shows results for the CSort strategy at 2 and 4 A100 GPUs running 32 rounds per batch.

Table 8: A100 decode, CSort, 32 rounds/batch, avg ms per round.

T	Dist	GPUs	Serial	Overlap
16	Uniform	2	2.510	2.241 (–10.7%)
32	Uniform	2	3.380	3.075 (–9.0%)
64	Zipf	2	3.976	3.587 (–9.8%)
128	Zipf	2	4.566	4.184 (–8.4%)
256	Zipf	2	4.933	4.516 (–8.5%)
16	Uniform	4	1.996	1.451 (–27.3%)

Overlap consistently helps in decode mode, unlike in large prefill batches. At 2 GPUs, overlap saves 8–11% depending on batch size. At 4 GPUs, the benefit grows to 27% at $T = 16$. The reason is that decode batches are small, so the FFN per round is short (1–4 ms) while the CPU sync step costs a fixed amount. In the prefill experiments with large batches, the CPU sync stall was large relative to the communication overlap window. In decode mode with $T = 16$ and 4 GPUs, the EP overhead fraction is 27.3% of the total round, so hiding communication behind computation yields real savings even with the CPU sync overhead.

3.8 Analysis

Several patterns summarize the results across all experiments.

Scatter optimization is necessary but not sufficient. Reducing scatter from 1.28 ms to 0.10 ms is a 12.7x improvement, but it only produces a 1.22x end-to-end speedup on V100 because FFN dominates. Under Zipf on A100, CSort with cuBLAS FFN gives only a modest 1.15× improvement over the Atomic baseline because the FFN time is amplified by load-imbalanced cuBLAS sequential calls; switching to grouped GEMM is required to translate scatter savings into a meaningful end-to-end speedup. The grouped GEMM is needed to realize the full benefit.

Grouped GEMM wins under Zipf. Sequential cuBLAS makes 64 separate GEMM calls, each with fixed launch overhead regardless of expert group size. Under Zipf, many experts are tiny and launch overhead dominates. A single grouped GEMM kernel handles all 64 experts together and is 1.22x faster than sequential cuBLAS under Zipf. Under uniform assignment the gain is smaller (1.07–1.13x) because all expert groups are roughly equal-sized and cuBLAS is well-utilized.

Blocksparse FFN is not practical. The blocksparse kernel is up to 31x slower than packed cuBLAS at $T = 2048$ Zipf. Avoiding the scatter step does not compensate for the FFN throughput loss. The irregular access pattern prevents the Tensor Cores from reaching high utilization.

Pipeline overlap works at decode scale, not prefill scale. In large prefill batches (V100, $T \geq 256$, 2–4 GPUs), overlap made performance 3–6% worse. In decode mode (A100, $T \leq 256$, 4 GPUs), overlap saved up to 27%. The difference is that decode rounds are short enough that the communication time is a meaningful fraction of total time, so hiding it behind computation helps. In long prefill rounds, the fixed CPU sync cost dominates and erases the benefit.

Scaling to more GPUs makes EP overhead more important. At 4 GPUs, each GPU's FFN is half the size of the 2-GPU case, but scatter and communication do not shrink proportionally. EP overhead at 4 GPUs with Atomic baseline reaches 27% of round time. This means the combined scatter-plus-grouped-GEMM optimization becomes more impactful at higher EP degree.

4 Conclusion

This project shows that optimizing MoE expert parallelism requires addressing both irregular token movement and expert computation. Scatter alone is highly optimizable: replacing the Atomic baseline with CSort reduces scatter latency by 12.7x on V100 by reducing atomic contention, improving memory coalescing, and avoiding unnecessary HBM passes. However, the end-to-end gain from scatter alone is limited because the FFN remains the dominant stage.

On A100, the FFN bottleneck becomes especially visible under skewed routing, where many experts receive small token groups and sequential per-expert cuBLAS calls underutilize the GPU. CUTLASS grouped GEMM reduces this overhead by executing many expert matmuls in a single kernel launch. Combined with CSort and overlap, grouped GEMM produces the best end-to-end prefill result.

The strong-scaling experiment shows that the optimized EP pipeline gives real multi-GPU acceleration, not only weak-scaling throughput growth. At fixed $T = 8192$, the optimized 8-GPU pipeline is 5.03x faster than the 1-GPU Atomic baseline under uniform routing. The same experiment reaches only 1.92x under Zipf routing, which identifies load imbalance as the main remaining bottleneck. In future work, this suggests that routing-aware load balancing, expert replication, or dynamic expert placement may be needed in addition to faster scatter and FFN kernels.

5 List of Work and Credit

Both team members contributed equally to this project, including problem identification and bottleneck analysis, overall system design, code framework construction, testing, and the writing of this report and poster. Feiyang Wu more focused on the end-to-end benchmark suite and the A100 experiments, while Yuhang Hu more focused on the Expert Parallelism benchmark and the V100 experiments. Credit is split 50%–50%.

References

[1] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. Megablocks: Efficient sparse training with mixture-of-experts, 2022.

[2] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding, 2020.

[3] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, 2017.

[4] Shawn Tan, Yikang Shen, Rameswar Panda, and Aaron Courville. Scattered mixture-of-experts implementation, 2024.

[5] Chenggang Zhao, Shangyan Zhou, Liyue Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. Deepep: an efficient expert-parallel communication library. https://github.com/deepseek-ai/DeepEP, 2025.